TrustVault: Trustless, Scalable, and Low-Cost On-Chain Academic Credential Verification Using Internet Computer Protocol

Varun Gudamsetti
Department of Computer Science and Engineering
PES University, Bengaluru, Karnataka, India
Email: varun.gudamsetti@pes.edu

Dr. Shruti Jadon *(Guide)*
Department of Computer Science and Engineering
PES University, Bengaluru, Karnataka, India
Email: shrutijadon@pes.edu

***Abstract*—Academic credential fraud imposes significant costs on institutions and employers worldwide. Existing solutions rely on either centralized registries or permissioned blockchains, both of which retain trust dependencies and single points of failure. This paper presents *TrustVault*, a fully on-chain academic credential verification system deployed on the Internet Computer Protocol (ICP) mainnet (canister `pekzw-diaaa-aaaad-afh3q-cai`). TrustVault stores credential data and its serving frontend inside ICP canisters and uses ICP *Certified Variables* — backed by threshold BLS signatures from a decentralized subnet — for mathematically verifiable, zero-trust authentication. An incremental binary Merkle tree provides O($\log n$) update cost as the credential database scales. Live mainnet benchmarks show issuance latency of ≈1.9 s, verification queries as fast as 177 ms, parallel throughput up to 7.3 certs/s, concurrent mixed-load throughput of 13.6 ops/s, and issuance cost of ≈$0.001 per certificate.**

***Index Terms*—Internet Computer Protocol, Academic Credential Verification, Certified Variables, Merkle Tree, Zero-Trust, Motoko, Threshold BLS Signatures**

I. Introduction

Academic credential fraud is a well-documented, escalating problem: studies estimate that a significant fraction of professional job applications misrepresent qualifications [1], and traditional verification — contacting institutions manually — is slow, expensive, and centralized. Blockchain-based alternatives have been proposed but each carries practical limitations. Bitcoin and Ethereum anchor credential hashes at prohibitive gas fees (tens of dollars per write), and the verification UI remains on a central server. Permissioned ledgers such as Hyperledger Fabric reduce cost but reintroduce consortium trust [14]. Hybrid systems storing only hashes on-chain with data off-chain restore centralization through the back door.

The *Internet Computer Protocol* (ICP) [2] offers a qualitatively different model. ICP runs smart contracts called *canisters* — WebAssembly modules replicated across independent *subnets* of geographically diverse nodes. Consensus achieves finality in ≈1–3 s via a BFT notarization protocol backed by the *Random Beacon* (threshold BLS over previous round outputs). Two protocol features are critical to this work: *on-chain web serving* (canisters respond to HTTP, eliminating cloud hosting) and *Certified Variables* (a native API where a canister commits a 32-byte value that the subnet collectively signs with a threshold BLS key [3], enabling data verification with no trust in the canister or any single node). Query calls — read-only operations bypassing consensus — return in <500 ms at no cost to the caller.

Contributions:

- TrustVault — the first fully on-chain academic credential system (frontend *and* backend) deployed on ICP mainnet.
- An incremental Merkle tree with O($\log n$) update cost, integrated with Certified Variables for trustless proof generation.
- A rigorous live mainnet evaluation: issuance, verification, throughput, finality, and concurrent mixed-load.
- A cross-platform comparison against Ethereum PoS [12] and Hyperledger Fabric [14] using only published benchmarks.

II. System Design

A. Architecture

The Motoko backend is organized into five modules: **main.mo** (public actor — `issueCertificate`, `revokeCertificate`, `verifyCertificate`), **CertificateManager.mo** (record hashing and management), **MerkleTree.mo** (incremental binary Merkle tree), **Utils.mo** (SHA-256, hex encoding), and **PerformanceMetrics.mo** (telemetry). The frontend is a React + Vite single-page application inside an ICP asset canister, served directly over HTTPS from the blockchain with no external hosting. It communicates with the backend through auto-generated Candid type bindings. Fig. 1 illustrates the full system.

Access control is enforced via ICP *Principals* — Ed25519-derived cryptographic identities. Only registered university principals may call `issueCertificate` or `revokeCertificate`; `verifyCertificate` is a free public query. Each certificate (Table I) carries a SHA-256 hash of its core identity fields and a consensus-set nanosecond timestamp. Stable Motoko variables persist the certificate map and Merkle root across canister upgrades.

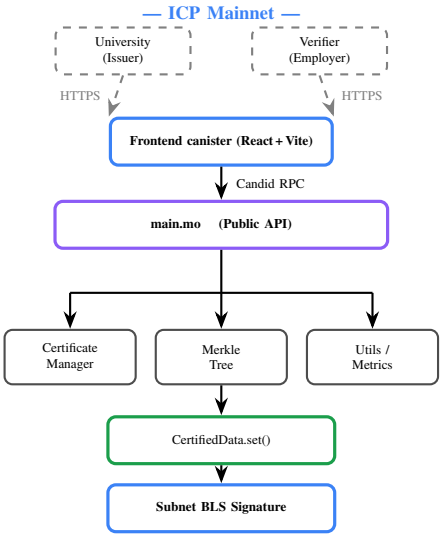

Fig. 1. TrustVault system architecture. University and Verifier access the on-chain Frontend canister via HTTPS. The Frontend dispatches Candid RPC calls to `main.mo`, which fans out to sub-modules via a shared dispatch bus. The MerkleTree module commits the certified root via `CertifiedData.set()`, BLS-signed by the subnet each consensus round.

TABLE I
CERTIFICATE SCHEMA

Field	Type	Description
`certificate_hash`	Text	SHA-256 of core fields
`issuer`	Record	University name, canister ID
`recipient`	Record	Student name, ID, principal
`credential`	Record	Degree type, major, GPA
`is_revoked`	Bool	Revocation flag
`block_timestamp`	Int64	Nanoseconds since Unix epoch

B. Incremental Merkle Tree and Certified Data

All certificate hashes are leaves of a complete binary Merkle tree stored as a flat array of size $2C-1$, where C is the smallest power of two $\geq$ the certificate count. Leaf i is at index $C-1+i$; the root is at index 0. Internal node j satisfies:

$$\texttt{nodes}[j] = H(\texttt{nodes}[2j+1] \parallel \texttt{nodes}[2j+2]) \tag{1}$$

Inserting a new leaf recomputes only the $O(\log n)$ path to the root (Fig. 2). After each issuance, the 32-byte Merkle root is committed with `CertifiedData.set()`, and the subnet threshold BLS-signs it within the next consensus round. Revocation triggers a full O(n) rebuild; the new certified root invalidates all pre-revocation proofs within one consensus round ($\approx$2 s).

III. TRUSTLESS VERIFICATION

Verification requires no trust in the issuing institution. Given a certificate and its Merkle proof π, the verifier: (1) recomputes $h_c = \text{SHA256}(\text{cert fields})$ and checks it matches the stored `certificate_hash`; (2) walks the Merkle proof to obtain the claimed root $\hat{R} = \text{verifyProof}(h_c, \pi)$; (3) fetches the ICP

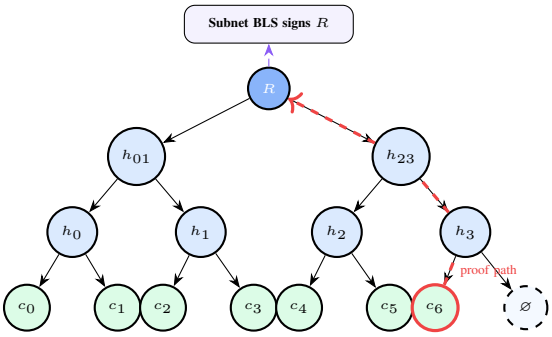

Fig. 2. Incremental Merkle tree. Leaf c_6 (red border) has proof path $[l_7,\ h_2,\ h_{01}]$ — three sibling hashes to recompute root R. The subnet BLS-signs R each consensus round.

TABLE II
PARALLEL ISSUANCE BENCHMARK — ICP MAINNET (3 TRIALS EACH)

Batch (n)	Wall mean (ms)	p95 (ms)	Thpt (cps)	Stddev
1	1943	2111	0.52	143.6
10	2596	3054	4.02	498.1
50	6933	7320	7.23	325.2
100	13969	15416	7.31	1926.5

Certificate (authenticated state tree + BLS witness) from any ICP gateway; (4) verifies the subnet's threshold BLS signature over the state tree root using the publicly known subnet key; (5) confirms the canister's certified value in the state tree equals $\hat{R}$.

The complete chain of trust is:

$$\text{cert fields} \xrightarrow{\text{SHA-256}} h_c \xrightarrow{\pi} \hat{R} \xrightarrow{\text{state tree}} \text{BLS sig} \xrightarrow{\text{subnet key}} \checkmark \tag{2}$$

Steps 1–2 execute client-side with no network call. Step 3 requires a single HTTP query to any ICP gateway. The subnet public key is published by the DFINITY Foundation and independently verifiable via the on-chain NNS — no step requires contacting the issuing institution.

IV. PERFORMANCE EVALUATION

All benchmarks were run against the live ICP mainnet canister `pekzw-diaaa-aaaad-afh3q-cai` in March 2026 using a Node.js benchmark suite (`@dfinity/agent`) from India ($\approx$50–80 ms RTT to ICP boundary nodes). Each metric was collected over 3 trials; we report min, mean, p50, p95, and p99.

A. Issuance Latency

Table II reports wall time and throughput for parallel issuance batches. At n=1, wall time mirrors one consensus round ($\approx$1.9 s). Parallelism amortizes the consensus overhead across all certificates in a batch: at n=50, throughput reaches 7.23 certs/s. Fig. 3 shows per-certificate latency percentiles; the distribution is tight for small batches and spreads at n=100 due to O(n) queue depth within a single consensus round.

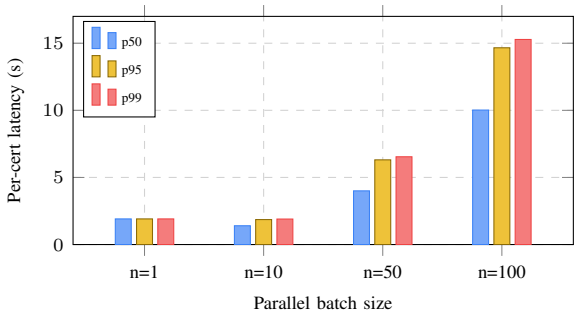

Fig. 3. Issuance latency percentiles (p50/p95/p99) per parallel batch size. Tight distribution at $n \leq 10$ confirms ICP round-trip dominance; spread at $n=100$ reflects queue-depth growth.

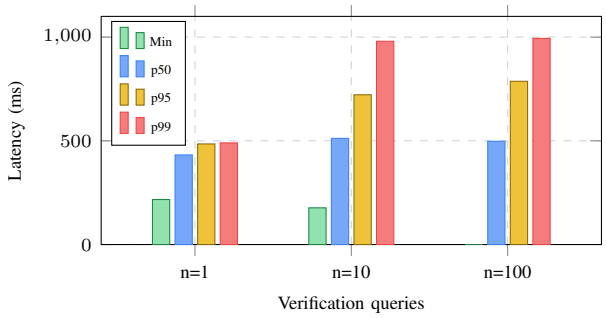

Fig. 4. Verification latency (min/p50/p95/p99) for 1, 10, and 100 concurrent queries against a 500-certificate database. Min bar omitted for $n=100$ due to a timing-measurement artifact.

TABLE III
VERIFICATION LATENCY — ICP MAINNET, 500-CERT DATABASE

Queries	Min	Mean	p50	p75	p95	p99
1	217	380	432	462	485	490
10	177	518	512	578	722	980
100	—	491	498	567	787	994

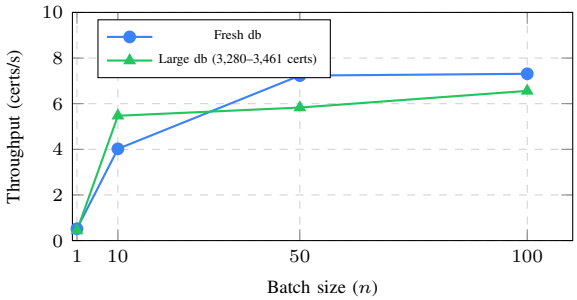

Fig. 5. Issuance throughput vs. batch size under fresh and large database conditions. The near-identical curves confirm $O(\log n)$ scalability independent of dataset size.

B. Verification Latency

Verification is a *query* call — no consensus, no fee. Table III and Fig. 4 report latency over a live database of 500 certificates. Minimum observed latency was **177 ms**, confirming that Merkle proof computation adds negligible overhead. The p99 remains below 994 ms even for 100-query bursts, making it suitable for real-time employer verification portals.

C. Throughput and Scalability

Fig. 5 overlays issuance throughput under two conditions: a fresh (low-count) database and a large database (≈3,280–3,461 existing certificates). The two curves nearly coincide, confirming $O(\log n)$ tree-insert scalability independent of database size. Peak throughput of **7.3 certs/s** is achieved at batch size 50. Over a 30-second sequential run, sustained throughput was stable at 0.50–0.60 certs/s with no degradation. A peak burst of 100 simultaneous certificates completed at 6.69 certs/s with **0% error rate**.

D. Concurrent Mixed-Load

The concurrent benchmark submits interleaved issuance and verification calls at concurrency level $c \in \{1, 5, 10, 25, 50\}$. Fig. 6 shows total throughput (ops/s, counting both call types) growing from 0.55 at $c=1$ to **13.59 ops/s** at $c=50$ — a ≈25× speed-up with **zero errors** at all levels. Fig. 7 shows verification latency (query, no consensus) remains stable in the 565–847 ms range across all concurrency levels, while issuance latency grows gracefully from 1.7 s to 2.8 s.

E. Finality Time and Cross-Platform Comparison

Table IV reports block finality time over 20 independent update calls. Mean finality is **2.36 s** with p99 under 2.95 s — a tight worst-case bound suitable for interactive applications. Table V places these results against published benchmarks for Ethereum PoS and Hyperledger Fabric; all non-ICP figures are from cited literature. Ethereum finality (≈24 s) is specified in [12]; network throughput (15–30 TPS) is from BLOCKBENCH [13]; gas costs for a credential write are derived from the Ethereum Yellow Paper [11]; Fabric throughput (≈3,500 TPS) is from Androulaki et al. [14]. ICP query calls (verification) are free to the caller; only update calls (issuance) consume cycles [3].

V. SECURITY ANALYSIS

TrustVault's security properties follow from the construction. **Data integrity**: any modification to a stored certificate changes h_c, breaks the Merkle proof, and causes a root mismatch detectable client-side without contacting the issuer. **Tamper-proof root**: the threshold BLS key is secret-shared; forging a valid subnet signature requires compromising >1/3 of subnet nodes simultaneously — computationally and logistically infeasible across a geographically diverse, independently operated network. **Replay resistance**: each certificate has a unique ID and a consensus-set nanosecond timestamp; duplicate issuance attempts are rejected. **Unauthorized issuance**: `issueCertificate` is gated by registered Ed25519-derived Principals; forging access requires the university's private key. **Revocation freshness**: the Merkle root is updated within one consensus round (≈2 s) of revocation, invalidating stale proofs far faster than traditional CRL re-

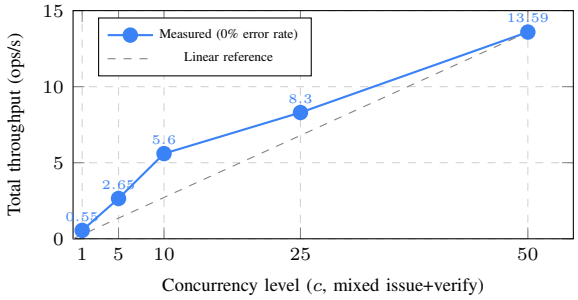

Fig. 6. Concurrent mixed-load throughput vs. concurrency. Sub-linear growth is due to update-call serialisation per consensus round; verification queries scale without contention.

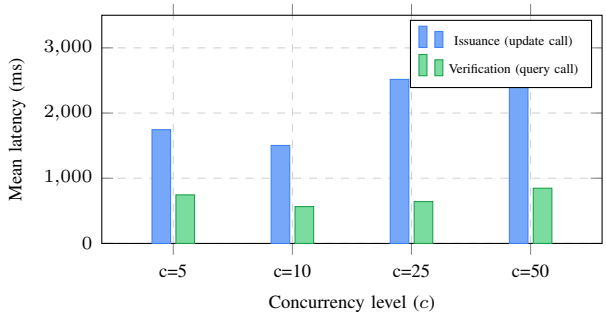

Fig. 7. Mean issuance vs. verification latency at each concurrency level. Query-based verification (no consensus) is ≈3× faster than update-based issuance at all levels.

fresh cycles. **Frontend integrity**: ICP's HTTP gateway signs canister asset responses via certified HTTP headers, making frontend tampering detectable in transit.

VI. Related Work and Conclusion

Related Work. Blockcerts [7] was the first system to anchor credential hashes on a public blockchain (Bitcoin), but verification still requires the issuer's web server, and the UI is centralized. EduCTX [8] uses a permissioned blockchain and requires trust in the participating consortium. Ethereum-based systems [9] store only hashes on-chain, keeping metadata off-chain and restoring centralization. W3C Verifiable Credentials [10] define a standard data model for credentials, but most implementations use permissioned ledgers or off-chain storage. None of the above systems serve the verification frontend from the blockchain, nor have been empirically evaluated on a public mainnet deployment.

Conclusion. TrustVault demonstrates that ICP's Certified Variables and on-chain web serving enable a practical, fully trustless academic credential system. Live mainnet benchmarks confirm: single-certificate issuance in ≈1.9 s; verification queries in 177–994 ms; parallel throughput of 7.3 certs/s; concurrent mixed-load throughput of 13.6 ops/s at 0% error rate; block finality of 2.36 s (p99 < 2.95 s); and an issuance cost of ≈$0.001 — orders of magnitude cheaper than Ethereum. Future work includes W3C Verifiable Credentials integration for cross-platform interoperability, multi-canister sharding for higher throughput, and Internet Identity integration for user authentication.

TABLE IV
Block Finality Time — ICP Mainnet (20 samples)

	Min	Mean	Std	p50	p75	p95	p99
ms	832	2356	421	2395	2622	2811	2944

TABLE V
Cross-Platform Comparison (non-ICP values from cited literature)

Metric	ICP (TrustVault)	Ethereum PoS	H. Fabric
Finality	**2.36 s** (meas.)	≈24 s [12]	<1 s [14]
Throughput	7.3 certs/s	15–30 TPS [13]	≈3500 TPS [14]
Issuance cost	**$0.001** (meas.)	$0.20–$2.00 [11]	Near-zero
Verification cost	**$0** (query)	$0.05–$0.50 [11]	Near-zero
Frontend on-chain	**Yes**	No	No
Trust model	**Zero-trust**	Zero-trust	Permissioned

References

[1] H. Chahal and S. Singh, "Credential fraud in South Asia: Findings and implications for the higher education sector," *Int. J. Educational Management*, vol. 35, no. 7, pp. 1498–1512, 2021.

[2] T. Hanke, M. Movahedi, and D. Williams, "DFINITY Technology Overview Series, Consensus System," *arXiv:1805.04548*, 2018.

[3] DFINITY Foundation, "The Internet Computer for Geeks," White Paper, 2022. [Online]. Available: https://internetcomputer.org/whitepaper.pdf

[4] V. Buterin, "A next-generation smart contract and decentralized application platform," *Ethereum White Paper*, 2014. [Online]. Available: https://ethereum.org/whitepaper

[5] R. C. Merkle, "A certified digital signature," in *CRYPTO'89*, LNCS vol. 435, pp. 218–238, 1989.

[6] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the Weil pairing," in *ASIACRYPT 2001*, LNCS vol. 2248, pp. 514–532, 2001.

[7] J. Grech and A. F. Camilleri, "Blockchain in Education," EUR 28778 EN, Publications Office of the EU, Luxembourg, 2017.

[8] M. Turkanović et al., "EduCTX: A blockchain-based higher education credit platform," *IEEE Access*, vol. 6, pp. 5112–5127, 2018.

[9] D. Lizcano et al., "Blockchain-based approach to create a model of trust in open and ubiquitous higher education," *J. Computing in Higher Education*, vol. 32, pp. 109–134, 2020.

[10] M. Sporny et al., "Verifiable Credentials Data Model v1.1," W3C Recommendation, Mar. 2022. [Online]. Available: https://www.w3.org/TR/vc-data-model/

[11] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Yellow Paper*, 2014.

[12] V. Buterin et al., "Combining GHOST and Casper," *arXiv:2003.03052*, 2020.

[13] T. T. A. Dinh et al., "BLOCKBENCH: A framework for analyzing private blockchains," in *Proc. ACM SIGMOD*, 2017, pp. 1085–1100.

[14] E. Androulaki et al., "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in *Proc. EuroSys'18*, 2018, pp. 1–15.